COMPTIA PENTEST+ · PT0-003 · OBJECTIVES V3.0

Domain 2.0 / Reconnaissance & Enumeration

A memorization-first study guide: what to learn, in what order, and the hooks to make it stick.

21%	~19	4	#1	2nd
of the exam	of 90 questions	objectives (2.1–2.4)	source of performance-based questions	heaviest domain, behind 4.0 (35%)

COLOUR KEY USED THROUGHOUT

PASSIVE no packet ever touches the client's asset · ACTIVE you interact with the target directly · BOTH depends on the mode or flag you choose

This one distinction decides more Domain 2 questions than any other idea in the domain. Learn the colour of every tool.

01 The shape of the domain — what each objective actually asks

02 The backbone: active vs. passive — the Touch Test, cue words, full tool classification

03 Nmap — flags, port states, NSE, reading output

04 Port → service → enumeration move — the muscle-memory table

05 OSINT sources — what each one yields

06 DNS — records, zone transfer, subdomain discovery

07 Web, WAF, and the origin address

08 Secrets, permissions, and attack-path mapping

09 Wireless enumeration

10 Objective 2.3 scripting — read, don't write

11 Look-alike tools — one line each, so distractors can't catch you

12 A 14-day study plan and how to practise

13 Rapid-recall drill — 32 prompts, fold-over format

14 One-page cheat sheet

01

The shape of the domain

Four objectives, four different question styles. Recognise the style and you know what kind of recall you need.

OBJ.	TITLE	WHAT IT REALLY TESTS	QUESTION STYLE
2.1	Apply information gathering techniques	Can you pick the right technique for the constraint you were given — especially *passive vs. active*	Short scenario, one best answer
2.2	Apply enumeration techniques	Given an open port or a service, what do you pull next	Scenario + output snippet; drag-and-drop PBQs
2.3	Modify scripts for recon and enumeration	Read a Bash / Python / PowerShell snippet and predict, fix, or complete it	PBQ: fill in the blank, fix the line
2.4	Use the appropriate tools	Straight tool recall, with near-neighbour distractors	One-liner multiple choice

HOOK — THE FOUR OBJECTIVES AS ONE SENTENCE

Find it (2.1) → open it up (2.2) → automate it (2.3) → with the right tool (2.4). Recon widens the picture; enumeration deepens it. If the question is about *discovering that something exists*, you're in 2.1. If it's about *listing what's inside something you already found*, you're in 2.2.

Where the marks actually are

- **Nmap alone** is worth studying as its own unit — it appears in 2.1, 2.2, 2.4, and again in Domains 3 and 4.
- **Output reading beats memorising commands.** The exam shows you results far more often than it asks you to type a command.
- **Domain 2 and Domain 3 overlap heavily** (17% more of the exam). Study them together — enumeration output is the input to vulnerability validation, and the tool lists share entries (Nmap, Wireshark, TruffleHog, BloodHound).
- Together, 2.0 + 3.0 = **38% of the exam**. That's more than Attacks and Exploits.

02

The backbone: active vs. passive

Learn this first. It converts a large family of questions into a two-way sort.

HOOK — THE TOUCH TEST

Did a packet of mine reach the client's asset? Then it's active. Everything else is passive.
Shodan is passive not because it's read-only, but because *Shodan* did the scanning, months ago, on its own dime. You're reading a database.

Cue words in the question → the answer type

IF THE SCENARIO SAYS…	THEY WANT…	TYPICAL RIGHT ANSWERS
"before the engagement begins", "no authorisation yet", "pre-engagement"	**Passive**	WHOIS, Wayback Machine, CT logs, Shodan/Censys, theHarvester, job boards
"without alerting the client", "must not be detected by the SOC", "zero footprint"	**Passive**	OSINT, public records, network sniffing of traffic already reaching you
"minimise the chance of detection" *but you must scan*	**Active, stealthy**	`nmap -sS -T1 -f -D`, fewer ports, longer intervals
"confirm", "validate", "verify the service version"	**Active**	Banner grab, `nmap -sV`, connect to the port
"the client's ICS/SCADA network", "production OT devices"	**Passive**	Port mirroring + manual assessment; never fuzz or actively scan live OT
"enumerate users on the domain controller"	**Active**	LDAP/SMB/Kerberos enumeration — you're authenticated or at least talking

TRAP COMPTIA LOVES

"Passive" does **not** mean "legal", "safe", or "unauthenticated". It means *no interaction with the target*. A fully authorised, credentialed scan is still active. Reading a leaked password dump is still passive.

Every objective 2.4 tool, sorted

TOOL	MODE	ONE-LINE IDENTITY
Wayback Machine	Passive	Historical snapshots of the site — dead endpoints, old admin paths, comments left in source
WHOIS	Passive	Domain registration: registrar, creation/expiry dates, name servers, sometimes contact email
Shodan	Passive	Search engine of internet-exposed devices and banners; the go-to for IoT/ICS discovery
Censys.io	Passive	Same idea as Shodan, strongest on certificates and TLS/host attribute search
Hunter.io	Passive	Corporate email addresses *and the naming convention* (first.last@, flast@)
theHarvester	Passive	CLI scraper: emails, names, subdomains from search engines and public sources
DNSdumpster	Passive	Web-based passive DNS recon with a host relationship map
OSINTframework.com	Passive	A directory *of* OSINT tools — not a scanner. Answer when asked "where do I find a tool for X?"
WiGLE.net	Passive	Crowdsourced database of wireless APs and their locations — wardriving without driving
Maltego	BOTH	Graph/link analysis; "transforms" pivot entity to entity. Visual relationship mapping
Recon-ng	BOTH	Modular CLI OSINT *framework* — workspaces, modules, API keys, Metasploit-style workflow
SpiderFoot	BOTH	Automated OSINT scanner, 200+ modules, web UI; fire once against a target and walk away
Amass	BOTH	OWASP subdomain enumeration; passive by default, active with brute force / zone walking
nslookup / dig	BOTH	DNS queries. Against a public resolver: passive-ish. Against the client's own name server: active
Wireshark / tcpdump	BOTH	Passive when sniffing traffic that already reaches your NIC; active once you're running it on their host
InSSIDer	BOTH	Windows Wi-Fi scanner: SSIDs, channels, channel overlap, signal strength. Site-survey tool
Nmap	Active	The scanner. Host discovery, ports, services, OS, and NSE scripts. *Exception:* `-sL` list scan only does reverse DNS
Nmap Scripting Engine	Active	Lua scripts layered on a scan: enumeration, brute force, vuln checks
Aircrack-ng	Active	Wireless suite. `airodump-ng` enumerates; the rest injects, deauths, and cracks

HOOK — THREE PASSIVE FAMILIES

Registries, Archives, Aggregators. Registries = WHOIS, DNS, CT logs. Archives = Wayback, cached pages, password dumps. Aggregators = Shodan, Censys, WiGLE, Hunter.io. If a tool fits one of those three buckets, it's passive.

03

Nmap — the highest-yield unit in the domain

If you learn one thing cold, learn this. Flags, port states, NSE, and above all reading the output.

Host discovery — "who is alive?"

FLAG	DOES	REMEMBER IT AS
`-sn`	Ping sweep: host discovery only, **no** port scan	"**n**o ports"
`-Pn`	Skip host discovery — treat every host as up. Essential when ICMP is blocked	"**P**ing **n**one"
`-sL`	List scan: resolves names, sends *nothing* to the target	The one passive Nmap mode
`-PS / -PA / -PU / -PE`	Discovery probe type: SYN / ACK / UDP / ICMP echo	Pick the probe a firewall lets through
`-iL targets.txt`	Read targets from a file	"**i**nput **L**ist"
`--exclude`	Honour scope exclusions from the rules of engagement	Domain 1 shows up here

Scan types — "what's open?"

FLAG	SCAN	KEY FACTS THE EXAM WANTS

`-sS`	SYN / half-open / "stealth"	**Default when run as root.** Never completes the handshake, so it's less likely to be logged by the application. Fast
`-sT`	TCP connect	Full three-way handshake. Noisier, gets logged, but **needs no privileges** — the answer when you're an unprivileged user
`-sU`	UDP	Slow and unreliable; no response ≠ closed. Where DNS 53, SNMP 161, TFTP 69 live
`-sA`	ACK	Maps *firewall rules*, not open ports. Distinguishes filtered from unfiltered
`-sN -sF -sX`	Null, FIN, Xmas	Flag-trickery scans to slip past simple stateless filters. Useless against modern Windows
`-sV`	Version detection	Interrogates the service banner → the version string you feed to CVE lookup
`-O`	OS detection	TCP/IP stack fingerprinting. **Needs at least one open and one closed port** to be reliable
`-A`	Aggressive	Equals `-sV -O -sC --traceroute`. Loud. Wrong answer whenever stealth is mentioned
`-p-` / `-p 1-65535`	All ports	Default without `-p` is the top **1,000** ports, not all of them
`--top-ports N`	N most common ports	The time-vs-coverage compromise answer

> **HOOK — THE THREE YOU MUST NOT MIX UP**
> **S**YN is **S**tealthy · **T**CP connect is **T**alkative · **U**DP is **U**nreliable.
> And: *lowercase `-sV` reads the service, uppercase `-O` reads the OS.*

Timing, output, evasion

FLAG	DOES
`-T0` … `-T5`	0 paranoid · 1 sneaky · 2 polite · 3 normal (default) · 4 aggressive · 5 insane. **T0–T1 are the IDS-evasion answers; T4 is the "I have a short window" answer**
`-f` / `--mtu`	Fragment packets so signature-based inspection can't reassemble the probe easily
`-D decoy1,ME,decoy2`	Decoy scan — bury your real source among spoofed ones
`-S ip` + `-e iface`	Spoof source address (you won't see replies)
`-g` / `--source-port 53`	Source-port manipulation: many weak firewalls trust traffic "from DNS"
`--data-length`	Pad packets so they don't match a default-size signature
`--spoof-mac`	Change MAC — useful on a local segment with MAC filtering
`--scan-delay` / `--max-rate`	Throttle to stay under rate-based alerting thresholds
`-oN` / `-oX` / `-oG` / `-oA`	**N**ormal · **X**ML (feeds Metasploit, reporting tools) · **G**reppable · **A**ll three at once

> **HOOK — TIMING LEVELS**
> "**P**aranoid **S**neaks **P**olitely, **N**ormally **A**ggressive **I**nsanity." T0 → T5 in order.

Port states — read these correctly and you'll win PBQs

STATE	MEANS	SAYS WHAT ABOUT THE NETWORK
`open`	Something is listening and accepting	Your next enumeration target
`closed`	Reachable, but nothing listening (RST received)	**Host is up and no firewall is filtering that port**
`filtered`	No reply, or an ICMP unreachable — a device dropped the probe	Firewall / ACL / IPS in the path. Try another probe type or port
`unfiltered`	Reachable but open/closed can't be told apart (ACK scan only)	Firewall isn't stateful for that port
`open|filtered`	No response at all (UDP, Null, FIN, Xmas)	Silence. Absence of evidence, not evidence of absence
`closed|filtered`	Idle scan couldn't distinguish the two	Rare; recognise it, don't chase it

HIGHEST-VALUE DISTINCTION IN DOMAIN 2

closed tells you a packet got there and came back — the host is alive and unfiltered.
filtered tells you something in the middle ate your probe. One is a finding about the host, the other is a finding about the firewall.

NSE — the Scripting Engine

Scripts live in `/usr/share/nmap/scripts`. Run with `--script`, tune with `--script-args`.

CATEGORY	USE / CAUTION
`default` (= `-sC`)	Safe, fast, useful. The everyday choice
`discovery`	Pulls more information about hosts and services — the Domain 2 workhorse
`safe` / `intrusive`	Won't crash things / might. **Check the rules of engagement before `intrusive`**
`auth` / `brute`	Auth bypass checks / credential guessing (noisy, lockout risk)
`vuln` / `exploit`	Known-vuln checks / active exploitation. Crosses into Domains 3 and 4
`dos` / `fuzzer`	Will disrupt. Almost always out of scope — and a wrong answer for OT/ICS
`external`	Sends data to a third party (e.g. WHOIS lookups) — a scope/privacy consideration
`malware` / `version` / `broadcast`	Backdoor detection / version aids / local-segment discovery

NAMED SCRIPTS WORTH MEMORISING

SCRIPT	GIVES YOU
`smb-enum-shares`, `smb-enum-users`	Share names and local users over SMB
`smb-os-discovery`	OS, computer name, domain, workgroup
`dns-brute`, `dns-zone-transfer`	Subdomains by brute force; AXFR attempt
`http-enum`, `http-title`, `http-methods`	Known paths, page titles, allowed verbs (PUT/DELETE = a finding)
`http-waf-detect`, `http-waf-fingerprint`	Whether a WAF is present and which one
`ssl-enum-ciphers`, `ssl-cert`	Weak ciphers/protocols, and cert SANs (more hostnames!)
`snmp-brute`, `snmp-info`	Community strings and device detail
`krb5-enum-users`	Valid domain usernames against Kerberos
`ftp-anon`	Anonymous FTP login allowed
`rdp-ntlm-info`	Hostname, domain, and OS build over RDP

Reading output like the exam wants

```
$ nmap -sS -sV -O -p- -T2 --open 10.10.14.7

PORT     STATE    SERVICE       VERSION
22/tcp   open     ssh           OpenSSH 7.2p2 Ubuntu 4ubuntu2.8
139/tcp  open     netbios-ssn   Samba smbd 3.X - 4.X
445/tcp  open     netbios-ssn   Samba smbd 4.3.11-Ubuntu
3306/tcp filtered mysql
8080/tcp open     http          Apache Tomcat 8.5.5
Running: Linux 4.X
Network Distance: 2 hops
```

Four things a question could ask, and how you answer:

- **Which host is this?** Linux, and specifically Ubuntu — the OpenSSH and Samba banners both say so. *Version strings beat `-O` guesses.*
- **What's your next enumeration step?** 445 → `smbclient -L` / `enum4linux`. It's the richest port on the list.
- **Why is 3306 filtered, not closed?** A firewall dropped the probe. MySQL may well be running and only reachable internally — note it for pivoting, don't call it absent.

- **Which finding is most likely exploitable?** Tomcat 8.5.5 on 8080 — a named application version, which is exactly what you take to CVE/EPSS in Domain 3.

> **HOOK — THE RECON-TO-ATTACK CHAIN**
> **Port → Service → Version → CVE → Exploit.** Every scan line you read is walking that chain. If a question asks "what do you do with this output," the answer is almost always "move one link to the right."

04

Port → service → enumeration move

Objective 2.2 in one table. If you can name the port, the protocol, and one tool, most of these questions collapse.

PORT	SERVICE	WHAT ENUMERATING IT GIVES YOU	GO-TO MOVE
21	FTP	Anonymous file access, banner, sometimes the whole web root	`nmap --script ftp-anon`; `ftp host` → anonymous
22	SSH	Version → distro and patch level; supported algorithms	`nc host 22`; `ssh2-enum-algos`
23	Telnet	Cleartext credentials on the wire; device type from banner	Banner grab + sniff
25 · 465 · 587	SMTP	**Valid usernames**, internal hostnames, relay misconfig	`VRFY`, `EXPN`, `RCPT TO:`; `smtp-enum-users`
53	DNS	The whole internal namespace if AXFR is allowed	`dig axfr @ns target.com`
69/udp	TFTP	Router/switch config files — often with hashes in them. No auth	`tftp` → `get startup-config`
80 · 443	HTTP(S)	App surface, tech stack, cert SANs = more hostnames	robots.txt, sitemap.xml, Gobuster, Burp/ZAP spider
88	Kerberos	**This host is a domain controller.** Username validity	`krb5-enum-users`; kerbrute
110 · 143 · 993 · 995	POP3 / IMAP	Mail server type; credential validity testing surface	Banner grab
111	rpcbind	Which RPC services exist, including NFS	`rpcinfo -p host`
135	MSRPC	Endpoint mapper → the services a Windows host offers	Impacket `rpcdump.py`
137–139	NetBIOS	Hostname, workgroup/domain, logged-on users	`nbtscan`, `nmblookup -A`
161/udp	SNMP	**The jackpot:** interfaces, routes, ARP cache, running processes, local users	`snmpwalk -v2c -c public host` (try `public` / `private`)
389 · 636	LDAP(S)	Naming context, users, groups, OUs, service accounts	`ldapsearch -x -H ldap://host -s base`, then `-b` the DN
445	SMB	**Shares, users, RIDs, OS, password policy, signing state**	`smbclient -L //host -N`; `enum4linux -a`; `rpcclient -U "" -N`
1433	MSSQL	Instance names, versions, linked servers	`ms-sql-info`; PowerUpSQL
1521	Oracle TNS	SID names	`oracle-sid-brute`
2049	NFS	Exported shares and their export options	`showmount -e host`
3306 · 5432	MySQL / PostgreSQL	Version, allowed hosts, sometimes anonymous access	`mysql-info`; banner
3389	RDP	Hostname, domain, NLA state, certificate name	`rdp-ntlm-info`
5900	VNC	Occasionally a desktop with no authentication at all	`vnc-info`
5985 · 5986	WinRM	Remote PowerShell — a foothold once you have credentials	`http-title`; Evil-WinRM (Domain 5)
6379 · 27017	Redis / MongoDB	Unauthenticated key/collection listing is common	`redis-cli -h host info`
9100 · 515	JetDirect / LPD	Printers store LDAP service-account credentials in plain config	Web console; treat printers as soft targets

502 · 20000 · 102	Modbus / DNP3 / S7 (OT)	Register values, device identity	PASSIVE ONLY port mirroring + manual review
1883 · 5683	MQTT / CoAP (IoT)	Topic data, often unauthenticated	Sniff; Shodan for exposed brokers

HOOK — THE FIVE PORTS THAT CARRY THE MOST EXAM WEIGHT

53, 161, 389, 445, 88. DNS gives you the map, SNMP gives you the inventory, LDAP gives you the directory, SMB gives you the shares and users, Kerberos tells you which box is the DC. Learn these five before the long tail.

OS fingerprinting without a scanner

OBSERVED TTL	ALMOST CERTAINLY
64	Linux / Unix / macOS
128	Windows
255	Network device (Cisco, most routers/switches)

Other fingerprint signals: service banners, HTTP `Server:` header, SMB dialect, and case sensitivity of URLs (Windows web servers usually aren't).

HOOK — TTL

Linux is **L**ean (64) · **W**indows is **W**ide (128) · **R**outers **R**ule (255). Values you see will be slightly lower — subtract one per hop.

ICS / OT RULE — MEMORISE THE REASONING, NOT JUST THE ANSWER

Legacy PLCs and controllers can fall over from an ordinary port scan, and a stopped process line is a safety incident, not an IT ticket. So for industrial systems the objectives name only two approaches: **manual assessment** and **port mirroring** (SPAN a switch port and analyse the copy). Any answer containing "scan", "fuzz", or "brute force" against live OT is wrong.

05

OSINT sources — what each one actually yields

Objective 2.1 lists the sources but not the payoff. The payoff is what the questions test.

SOURCE	WHAT YOU TAKE FROM IT	FEEDS WHICH LATER PHASE
Social media	Org chart, job titles, who reports to whom, badge and lanyard photos, office layout in the background	Spearphishing and pretexting (4.8); tailgating (3.3)
Job boards	**Your version intelligence.** "Must have 3 years Citrix 7.15 and Splunk" names the stack and often the version	Target prioritisation (4.1)
Code repositories	Hardcoded credentials, API keys, internal hostnames, database schemas, CI config	Secrets enumeration; initial access
DNS / reverse DNS	Hosts, mail infrastructure, third-party services, naming conventions	Everything
Cached pages	Pages the client deleted, old admin endpoints, developer comments in source	Directory enumeration; web attacks (4.5)
Cryptographic flaws	Expired/self-signed certs, weak ciphers, and cert **SAN fields listing hostnames you hadn't found**	Vulnerability analysis (3.x)
Password dumps	Old credentials for the client's email domain, plus the password patterns their users prefer	Credential stuffing and spraying (4.3)
Certificate transparency logs	Every certificate ever issued for the domain — including internal and dev hostnames never meant to be public	Subdomain discovery; WAF origin hunting

SEARCH ENGINE ANALYSIS (GOOGLE DORKING)

OPERATOR	FINDS
`site:target.com`	Everything indexed for the domain — start here, then subtract
`site:target.com -www`	Subdomains other than the main site
`filetype:pdf` / `ext:xlsx`	Documents — and their metadata (authors, usernames, software versions)

`inurl:admin` / `intitle:"index of"`	Admin panels; open directory listings
`intext:"internal use only"`	Documents that were never meant to be indexed

> **HOOK — THE OSINT RULE OF THUMB**
> **People, Places, Platforms, Paper.** People = staff and emails. Places = IPs, domains, APs. Platforms = tech stack and versions. Paper = documents, dumps, and archived pages. Any OSINT answer belongs to one of those four.

06

DNS — small topic, disproportionate marks

DNS is the only place where one misconfiguration hands you the entire internal map.

Records and what they leak

RECORD	HOLDS	RECON VALUE
A / AAAA	Hostname → IPv4 / IPv6	The target list. IPv6 addresses are often less firewalled
NS	Authoritative name servers	Who to ask for a zone transfer
MX	Mail exchangers	Mail platform; often the real IP behind a CDN
TXT	SPF, DKIM, DMARC, verification strings	**SPF lists their mail senders and IP ranges**; verification records name their SaaS vendors
CNAME	Aliases	Third-party hosting; dangling CNAME = subdomain takeover
SOA	Zone authority and admin contact	Admin email, zone serial
SRV	Service locations	`_ldap._tcp.dc._msdcs` → domain controllers, by name
PTR	IP → hostname (reverse)	Naming conventions across a whole CIDR block

Zone transfer — the one command to have memorised

```
# 1. find the name servers
dig ns target.com +short
# 2. ask each one for the entire zone
dig axfr @ns1.target.com target.com
# same thing via nmap
nmap --script dns-zone-transfer --script-args dns-zone-transfer.domain=target.com -p 53 ns1.target.com
```

A successful AXFR returns every record in the zone: internal hosts, dev boxes, VPN endpoints, printers. It is a finding in its own right — an information-disclosure misconfiguration — and it should appear in the report even if you gained nothing from it.

Subdomain discovery, from quietest to loudest

MODE	METHOD	TOOL
Passive	Certificate transparency logs	crt.sh, Censys.io
Passive	Aggregated public DNS data	DNSdumpster, Amass (default mode), SpiderFoot
Passive	Search engine scraping	theHarvester, `site:` dorks
Passive	Reverse DNS across a CIDR, name resolution only	`nmap -sL 10.0.0.0/24`, `dig -x`
Active	Wordlist brute force against their name server	Amass (active), `dns-brute`, dnsrecon, Gobuster dns mode
Active	Zone transfer attempt	`dig axfr`

> **HOOK**
> **Certificates never lie about hostnames.** When a question asks how to find hosts *without touching the target*, certificate transparency is the answer more often than anything else on the list.

07

Web, WAF, and the origin address

Two items candidates routinely skip, and both are named explicitly in objective 2.2.

Manual enumeration first — it's free and it's silent

LOCATION	WHY YOU CHECK IT
`/robots.txt`	A list of the paths they'd rather you didn't visit. Never a control, always a hint
`/sitemap.xml`	Author-supplied inventory of pages, including ones not linked in the nav
`/.well-known/`	`security.txt` (a contact for your report), plus platform metadata
Page source & JS bundles	Comments, API endpoints, internal hostnames, embedded keys
HTTP response headers	`Server`, `X-Powered-By`, framework cookies (`JSESSIONID`, `PHPSESSID`, `ASP.NET_SessionId`) → the stack
Platform plugins	WordPress: `/wp-content/plugins/`, versions in `readme.txt`. **WPScan** automates it
Error pages	Stack traces name the framework, the path, and sometimes the database

Directory enumeration and status codes

```
gobuster dir -u https://target.com -w /usr/share/wordlists/dirb/common.txt -x php,bak,txt
ffuf -u https://target.com/FUZZ -w wordlist.txt        # FUZZ is the placeholder
wfuzz -c -z file,wordlist.txt --hc 404 https://target.com/FUZZ
```

CODE	READ IT AS
200	It exists and you can read it
301 / 302	It exists — follow the redirect, the target often reveals structure
401	It exists and wants credentials
403	**It exists but is blocked — a positive finding, not a dead end**
404	Nothing there (unless the app returns a soft 404 — check the response length)
406 / 419 / 429	You're probably talking to a WAF, not the app
500	You broke something — note it, it may be injectable

WAF enumeration → then find the origin

Detect it: `nmap --script http-waf-detect,http-waf-fingerprint`, `wafw00f https://target.com`, or by hand — look for a `cf-ray` / `x-cdn` / `x-sucuri` header, a vendor-branded block page, or a 406/429 in response to a benign payload.

Then find the origin address, because a WAF only protects what routes through it. If you can reach the web server directly you bypass the entire control.

TECHNIQUE	WHY IT WORKS
Certificate transparency logs	The origin's own cert was logged before the CDN was put in front of it
Historical DNS records	The A record pointed at the real IP before the migration
Non-proxied subdomains	`mail.`, `ftp.`, `dev.`, `vpn.` are usually left off the CDN and often sit in the same block
SPF and MX records	Mail can't be proxied, so it exposes real infrastructure IPs
Shodan / Censys search by cert or favicon hash	Finds the naked server that's still serving the same certificate
Verbose error pages	Stack traces and email headers leak internal IPs and hostnames

> **HOOK**
> **A WAF is a doorman, not a wall.** Enumerate the doorman, then look for the side entrance. Every origin-discovery technique above is passive — that's why this topic pairs so naturally with active vs. passive questions.

08

Secrets, permissions, and attack-path mapping

Objective 2.2 names secrets enumeration, permission enumeration, and attack path mapping. All three are quick wins.

Secrets enumeration — four things: what, where, how, so what

SECRET TYPE	WHERE IT HIDES	SHAPE YOU'D GREP FOR
Cloud access keys	Git history, CI logs, mobile app packages, `.env`	AWS `AKIA…` / `ASIA…`, Google `AIza…`
API keys / tokens	JS bundles, mobile apps, Postman collections	Slack `xox…`, GitHub `ghp_…`, long base64 strings
Passwords	Config files, scripts, documentation, SYSVOL/GPP, printer configs	`password=`, `connectionString`, `cpassword`
Session tokens	Client-side storage, URLs, logs	JWTs start `eyJ` — three dot-separated base64url parts

Tools: TruffleHog — the one to know; scans *git history*, not just the current checkout, so deleted-but-committed secrets still surface. Also exposed `/.git/` directories, public storage buckets, and CI build logs.

> **WHY THE EXAM CARES**
> A found credential turns recon straight into initial access, skipping exploitation entirely. When a question offers a credential-discovery answer alongside an exploit answer, the credential is usually both quieter and more likely to succeed.

Permission enumeration

PLATFORM	COMMANDS WORTH RECOGNISING
Windows	`whoami /all`, `icacls`, `accesschk.exe`, `Get-Acl`, `net user /domain`
Linux	`id`, `sudo -l`, `find / -perm -4000 2>/dev/null` (SUID), world-writable files
Cloud	IAM policy review — Prowler, ScoutSuite (they reappear in objective 4.6)
Shares	Share permissions vs. NTFS permissions: the effective right is the *more restrictive* of the two

Attack path mapping

BloodHound is the answer whenever a question mentions Active Directory relationships, "shortest path", or "which account gets me to Domain Admin". A collector (SharpHound) gathers users, groups, sessions, and ACLs; BloodHound renders it as a graph and computes paths.

- Nodes: users, computers, groups, GPOs, OUs, domains.
- Edges are the exam-relevant part: `MemberOf`, `AdminTo`, `HasSession`, `GenericAll`, `WriteDacl`, `CanRDP`.
- Collection is ACTIVE and authenticated — it needs a domain account.

> **HOOK**
> **Nmap maps the network; BloodHound maps the trust.** Same job, different layer.

09

Wireless enumeration

Objective 2.2 lists wireless enumeration; 3.1 adds SSID, channel, and signal-strength scanning. Small topic, easy marks.

WHAT YOU ENUMERATE	WHY IT MATTERS
ESSID / BSSID	Network name and the AP's MAC. Two APs with one ESSID = either roaming or an evil twin
Channel	You must lock your card to a channel to capture reliably; overlap indicates a poorly planned deployment
Signal strength	**Site survey:** does usable signal leak into the car park? That's a physical-security finding
Encryption / auth	Open · WEP · WPA2-PSK · WPA2-Enterprise · WPA3 — decides which attack in 4.7 is even possible
Associated clients	Station MACs, and probe requests that reveal **hidden SSIDs** and networks clients have joined before

TOOL	MODE	IDENTITY

WiGLE.net	Passive	Crowdsourced AP location database — recon a site before you ever go there
Kismet	Passive	Pure passive sniffer/WIDS; finds hidden SSIDs, logs everything, multi-protocol
InSSIDer	BOTH	Windows GUI: SSIDs, channels, signal strength. The site-survey answer
Aircrack-ng suite	Active	`airmon-ng` monitor mode → `airodump-ng` capture/enumerate → `aireplay-ng` inject

HOOK — HARDWARE REQUIREMENT

Wireless work needs an adapter that supports **monitor mode and packet injection**, plus a directional antenna for range. The hardware list in the objectives is fair game for a question.

10

Objective 2.3 — read scripts, don't write them

The exam asks you to predict output, spot the broken line, or complete a blank. It never asks you to compose a program from nothing.

The same loop in all three languages

Learn one pattern — sweep a subnet — in Bash, Python, and PowerShell. Nearly every 2.3 question is a variation on it.

```
# BASH
for ip in $(seq 1 254); do
  ping -c 1 -W 1 10.0.0.$ip > /dev/null 2>&1 && echo "10.0.0.$ip is up"
done
```

```
# PYTHON
import socket
for port in range(20, 1025):
    s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    s.settimeout(1)
    if s.connect_ex(("10.0.0.5", port)) == 0:      # 0 means OPEN
        print(f"port {port} open")
    s.close()
```

```
# POWERSHELL
foreach ($i in 1..254) {
  if (Test-Connection -ComputerName "10.0.0.$i" -Count 1 -Quiet) {
    Write-Output "10.0.0.$i is up"
  }
}
```

Syntax comparison — the table to memorise

CONSTRUCT	BASH	PYTHON	POWERSHELL
Count loop	`for i in {1..10}` `for i in $(seq 1 10)`	`for i in range(1,11):`	`foreach ($i in 1..10)`
Loop over a file	`while read l; do … done < f`	`for l in open("f"):`	`Get-Content f | ForEach-Object { }`
Conditional	`if [ … ]; then … fi`	`if …: / elif / else`	`if (…) { } elseif { } else { }`
Numeric compare	`-eq -ne -gt -lt -ge -le`	`== != > < >= <=`	`-eq -ne -gt -lt -ge -le`
String compare	`= != -z -n`	`== != in`	`-eq -like -match -contains`
Boolean	`&& || !`	`and or not`	`-and -or -not`
Variable	`$name` (no space around `=`)	`name`	`$name`
Comment	`#`	`#`	`#`
Block ends with	`done` / `fi`	Indentation	`}`

Libraries, modules, and cmdlets the objectives expect

LANGUAGE	RECOGNISE THESE

Bash	`cut -d: -f1` · `awk '{print $1}'` · `grep -E` · `sort -u` · `uniq -c` · `tr` · `sed 's/a/b/g'` · `xargs` · `>/dev/null 2>&1`
Python	`socket` (connect_ex) · `requests` (HTTP) · `ipaddress` (iterate a CIDR) · `subprocess` (call a tool) · `re` · `json` · `scapy` (craft packets) · `impacket` (SMB/Kerberos)
PowerShell	`Test-NetConnection` · `Test-Connection` · `Resolve-DnsName` · `Get-ADUser` / `Get-ADComputer` · `Invoke-WebRequest` · `Get-Content` · `Select-String` · `New-Object System.Net.Sockets.TcpClient`

HOOK — PICK THE LANGUAGE BY THE TARGET

Bash for Linux hosts and gluing tools together. PowerShell for Windows and Active Directory. Python for anything that needs packets, parsing, or portability. If a question asks which language suits the task, that mapping answers it.

Spot-the-bug checklist

SYMPTOM IN THE SNIPPET	THE BUG
Sweep misses `.254`, or misses the last item	Off-by-one: `range(1,254)` stops at 253; `seq 1 254` includes 254
`if [$x -eq 1]`	Bash needs spaces *inside* the brackets: `if [ $x -eq 1 ]`
`if [ $x = 1 ]` used for a number	`=` compares strings, `-eq` compares numbers
Variable reads as empty in Bash	Missing `$`, or a space around the `=` at assignment
Python throws `TypeError` comparing input	`input()` returns a string — wrap in `int()`
Python `IndentationError`	Body of the loop or `if` isn't indented consistently
Scan hangs forever	No timeout set on the socket / no `-W` on ping
PowerShell loop prints nothing	Missing `$` on the loop variable, or `-Quiet` omitted so the object isn't a boolean
Missing `done` / `fi` / `}`	Unclosed block — always count the closers

HOW TO PRACTISE THIS

Type each of the three scripts above from memory, then deliberately break one line and fix it. Ten minutes a day for a week is worth more than reading a scripting chapter twice, because the exam gives you broken code, not blank pages.

11

Look-alike tools — the distractor defence

CompTIA builds wrong answers out of near neighbours. One sharp differentiating clause per tool is all you need.

CONFUSABLE PAIR	THE LINE THAT SEPARATES THEM
Shodan vs **Censys.io**	Both index exposed hosts. Shodan is the banner/IoT/ICS answer; Censys is the certificate and TLS-attribute answer
Maltego vs **Recon-ng** vs **SpiderFoot**	Maltego *draws* the relationships (graph + transforms). Recon-ng is a modular CLI framework *you drive*. SpiderFoot *runs itself* — automated, 200+ modules, web UI
theHarvester vs **Hunter.io**	theHarvester scrapes multiple sources from the CLI. Hunter.io is a web service specifically for email addresses and the naming convention
Amass vs **DNSdumpster** vs **dig**	Amass = thorough subdomain enumeration, passive or active. DNSdumpster = quick passive web lookup with a map. dig = one precise query at a time
dig vs **nslookup**	dig is scriptable and does AXFR, `+short`, `+trace`. nslookup is the interactive tool that's always present on Windows
WHOIS vs **DNS lookup**	WHOIS = who *registered* it (registrar, dates, contacts). DNS = where it *resolves*
Wayback Machine vs **cached pages**	Wayback = deep historical archive over years. Search-engine cache = the most recent copy only
WiGLE.net vs **InSSIDer** vs **Kismet**	WiGLE = someone else's wardriving database (no radio needed). InSSIDer = your Windows laptop surveying now. Kismet = passive sniffer that also finds hidden SSIDs

Wireshark vs **tcpdump**	Same captures, different ergonomics: Wireshark for GUI analysis and dissection, tcpdump for headless capture over SSH and quick filters
Nmap vs **Nessus/OpenVAS**	Nmap answers "what's there?". A vulnerability scanner answers "what's wrong with it?" (Domain 3)
Gobuster vs **Burp/ZAP spider**	Gobuster guesses paths from a wordlist (finds unlinked content). A spider follows links (finds linked content). You need both
OSINTframework.com	Not a tool — a categorised directory of tools. Correct answer to "where do I look for a tool that does X?"

12

A 14-day plan, and how to practise

Study Domain 2 with Domain 3, not before it. Enumeration output is the input to vulnerability analysis.

DAYS	FOCUS	DONE WHEN YOU CAN…
1–3	Nmap: flags, port states, NSE	Write a stealth scan, an all-ports scan, and an unprivileged scan from memory, and explain filtered vs. closed out loud
4–5	Port → service → enumeration table	Recite the move for 53, 88, 161, 389, 445 without looking
6	Active vs. passive sort	Classify all 19 objective-2.4 tools with no errors
7	OSINT sources and Google dorks	Name what each source yields *and* which later phase uses it
8	DNS: records, AXFR, subdomain discovery	Type the two-step zone-transfer sequence from memory
9	Web enumeration, WAF, origin discovery	List four ways to find an origin IP behind a CDN
10	Wireless + secrets + attack paths	Explain why WiGLE is passive and BloodHound isn't
11–12	Scripting: the three-language loop	Reproduce all three sweep scripts and fix five planted bugs
13	Look-alike tool drill	Give the one-line identity for every tool, cold
14	Mixed practice on output-reading questions	Read an unfamiliar scan and state the next three steps

Four practice habits that matter more than re-reading

1. **Do everything twice — by hand, then scripted.** Enumerate a host manually with `nc`, `dig`, and `smbclient`; then write the loop that does it for a whole subnet. That single exercise covers 2.1, 2.2, and 2.3 at once.
2. **Read output, don't just run commands.** Save your scan results and quiz yourself later on what each line implies. This is the exact skill the PBQs test.
3. **Keep a one-page tool sheet in your own words.** One line per tool. Rewriting the line is what fixes it; re-reading someone else's list doesn't.
4. **Narrate the engagement.** Say the chain out loud: OSINT → DNS → hosts → ports → services → shares and users → secrets → attack path. When a question drops you in the middle, you'll know which link you're on and therefore what comes next.

A lab that's enough

- Kali (or Parrot) as the attacker, on a host-only or NAT network.
- **Metasploitable 2/3** for the full port table — FTP, SMB, SNMP, MySQL, Tomcat all in one box.
- A Windows Server evaluation VM promoted to a domain controller, plus one workstation, for LDAP/Kerberos/SMB/BloodHound practice.
- Or skip the build: **TryHackMe** and **Hack The Box** both have recon and enumeration paths that mirror these objectives.

> **SCOPE DISCIPLINE IS ITSELF AN EXAM TOPIC**
> Only ever scan systems you own or are explicitly authorised to test. Domain 1.1 (authorisation letters, rules of engagement, exclusions) gets tested *through* Domain 2 scenarios — "the client's CIDR range is X, you notice Y in the results" questions are really asking whether you'd stop.

13

Rapid-recall drill

Cover the right column. Say the answer out loud before you look. Anything you miss twice goes on an index card.

Is querying Shodan passive or active?	Passive — Shodan did the scanning, you're reading its database
How many ports does Nmap scan with no `-p`?	The top 1,000 — not all 65,535

Which TCP scan works without root?	`-sT` connect scan (completes the handshake, so it gets logged)
Which flag assumes hosts are up when ICMP is blocked?	`-Pn`
Which port state means a device dropped your probe?	`filtered`
Which port state proves the host is alive and unfiltered?	`closed` — a RST came back
What does `-A` expand to?	`-sV -O -sC --traceroute` — and it's wrong whenever stealth is required
Which timing templates are the IDS-evasion answer?	`-T0` paranoid and `-T1` sneaky
Which output format feeds other tools?	`-oX` XML (`-oA` writes all three formats)
Type the zone transfer.	`dig axfr @ns1.target.com target.com`
Which DNS record reveals their mail-sending IP ranges?	TXT, via the SPF record
Which record type names domain controllers?	SRV — `_ldap._tcp.dc._msdcs.domain`
The two SNMP community strings you always try?	`public` and `private`
Port 88 is open. What have you found?	Kerberos → a domain controller → Active Directory
Port 445 is open. Name three things to pull.	Shares, local/domain users, OS and domain name (`smbclient -L`, `enum4linux -a`, `rpcclient`)
TTL 64 / 128 / 255?	Linux / Windows / network device
The one Nmap mode that sends nothing to the target?	`-sL` list scan — reverse DNS only
Best zero-contact way to discover hostnames?	Certificate transparency logs (crt.sh, Censys)
Gobuster returns 403. Dead end?	No — the path exists but is blocked. Report it and try around it
Four ways to find the origin behind a WAF?	CT logs · historical DNS · non-proxied subdomains (mail/dev/vpn) · SPF and MX records
Which tool finds secrets in *deleted* commits?	TruffleHog — it walks git history, not just the checkout
A token beginning `eyJ` is?	A JWT — three base64url segments separated by dots
AD attack-path mapping tool and its collector?	BloodHound, collected by SharpHound
The only two approaches for assessing live ICS?	Manual assessment and port mirroring — never active scanning or fuzzing
Which wireless recon needs no radio at all?	WiGLE.net — someone else already drove past
What must a wireless adapter support?	Monitor mode and packet injection
Bash numeric equality?	`-eq` (`=` compares strings)
Python call that returns 0 for an open port?	`socket.connect_ex((ip, port))`
PowerShell cmdlet to test a port?	`Test-NetConnection -ComputerName host -Port 445`
What does `range(1,254)` actually cover?	1 through 253 — the classic planted off-by-one
What is OSINTframework.com?	A directory of OSINT tools, not a tool itself
Gobuster vs. a spider?	Gobuster guesses unlinked paths from a wordlist; a spider follows links that exist
What do job boards give a pentester?	Version and stack intelligence — the technologies and versions they run
SMTP verbs for user enumeration?	`VRFY`, `EXPN`, `RCPT TO:`

14

One-page cheat sheet

Print this page on its own. It's the last thing to read before the exam.

NMAP, FIVE COMMANDS

- `nmap -sn 10.0.0.0/24` — who's alive
- `nmap -sS -p- -T4 host` — all ports, half-open
- `nmap -sV -sC -O host` — versions, default scripts, OS
- `nmap -sU --top-ports 20 host` — the UDP essentials
- `nmap -sS -T1 -f -D d1,ME,d2 host` — quiet

PORT STATES

- **open** — listening
- **closed** — host up, nothing listening, nothing filtering
- **filtered** — a firewall ate the probe
- **unfiltered** — ACK scan only; not stateful
- **open|filtered** — silence (UDP/Null/FIN/Xmas)

THE FIVE PORTS THAT PAY

- **53** DNS — the map (AXFR)
- **161** SNMP — the inventory (`public`)
- **389** LDAP — the directory
- **445** SMB — shares and users
- **88** Kerberos — this is the DC

ENUMERATION CHAIN — SAY IT OUT LOUD

OSINT → DNS → live hosts → ports → service versions → shares and users → secrets → attack path.

EVERY MNEMONIC IN THIS GUIDE

- **Touch Test** — my packet reached their asset ⇒ active
- **S**tealthy / **T**alkative / **U**nreliable — `-sS -sT -sU`
- **P**aranoid **S**neaks **P**olitely, **N**ormally **A**ggressive **I**nsanity — T0→T5
- **L**ean 64 / **W**ide 128 / **R**outers 255 — TTL
- **Registries, Archives, Aggregators** — the passive families
- **People, Places, Platforms, Paper** — OSINT buckets
- **Port→Service→Version→CVE→Exploit** — the chain
- **Nmap maps the network, BloodHound maps the trust**

CUE WORD → ANSWER TYPE

- "before authorisation", "no contact" ⇒ **passive OSINT**
- "undetected" + must scan ⇒ **-sS -T1 -f -D**
- "confirm the version" ⇒ **-sV / banner grab**
- "unprivileged account" ⇒ **-sT**
- "ICMP is blocked" ⇒ **-Pn**
- "production OT / SCADA" ⇒ **port mirroring, manual**
- "shortest path to Domain Admin" ⇒ **BloodHound**
- "find hosts without touching them" ⇒ **CT logs**
- "secrets in a repo" ⇒ **TruffleHog**
- "email naming convention" ⇒ **Hunter.io**
- "exposed IoT/ICS device" ⇒ **Shodan**

ONE-WORD TOOL IDS

- Shodan *devices* · Censys *certificates* · Wayback *history* · WHOIS *registration*
- Maltego *graph* · Recon-ng *framework* · SpiderFoot *automation*
- theHarvester *scraper* · Hunter.io *emails*
- Amass *subdomains* · DNSdumpster *quick DNS* · dig *precise* · nslookup *always there*
- WiGLE *database* · InSSIDer *survey* · Kismet *passive sniffer* · Aircrack-ng *suite*
- Wireshark *analyse* · tcpdump *capture*
- TruffleHog *secrets* · BloodHound *paths* · Nmap *everything*

SCRIPTING QUICK FACTS

- Bash: `-eq` numbers, `=` strings, spaces inside `[ ]` , close with `done` / `fi`
- Python: `range(a,b)` excludes `b` ; `connect_ex` returns 0 = open; indentation is syntax
- PowerShell: `1..254` , `-eq -like -match` , `Test-NetConnection` , close with `}`
- Choose by target: Bash = Linux, PowerShell = Windows/AD, Python = packets and parsing

DON'T LOSE EASY MARKS ON

- `filtered` vs `closed`
- Passive ≠ safe, legal, or unauthenticated
- Default scan is 1,000 ports, not all
- 403 in directory enumeration is a finding
- Never actively scan live OT
- Exclusions in the rules of engagement bind you even when a host looks interesting

PenTest+ PT0-003 · Domain 2.0 study guide. Objective structure, tool names, and terminology follow CompTIA's *PenTest+ PT0-003 Certification Exam Objectives, version 3.0*; the groupings, tables, mnemonics, and study plan are study aids built around them, not CompTIA material. Objective 2.4 lists 19 tools — every one appears in section 02. Practise only against systems you own or are authorised in writing to test.